PIMPRI CHINCHWAD EDUCATION TRUST's.

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

(An Autonomous Institute)

B. TECH **Year:** 2026 – 27 **Semester:** VII

Name of the Student: Kartik Thakur **PRN:** 123B1B067

Department: Computer Engineering **Division:** A

Course: Object Oriented Modelling and System Design **Course Code:** BCE27PC02

Assignment No – 7

1. Aim

To design a Smart Healthcare System using modern software architecture and UML deployment modeling techniques. The proposed system supports patient management, electronic health records (EHR), appointment scheduling, telemedicine, billing, healthcare analytics, AI-based disease prediction, wearable integration, and secure data exchange.

2. Objectives

- Apply software architecture principles to a healthcare information system.
- Design a scalable microservice-based enterprise architecture.
- Create a UML deployment model for a distributed cloud application.
- Identify major services, components, databases, and external integrations.
- Design security controls for sensitive healthcare information.
- Analyze scalability, availability, reliability, performance, and maintainability.
- Justify architectural decisions and identify key trade-offs.
- Understand cloud-native deployment strategies and asynchronous communication.

3. Course Outcomes Addressed

CO	Description
CO1	Apply object-oriented modeling principles.
CO2	Design UML diagrams for real-world systems.
CO3	Develop scalable software architectures.
CO4	Design secure and maintainable enterprise systems.

4. Problem Statement

A healthcare organization requires a Smart Healthcare System capable of supporting thousands of patients and healthcare professionals. The system must provide patient registration, profile and medical-history management, appointment scheduling, electronic health records, diagnosis and prescription management, online consultation, hospital resource and staff management, billing and payments, telemedicine, health analytics, AI-based disease prediction, and wearable-device integration. The solution must provide 24×7 availability, strong privacy and security, fast access to records, low-latency telemedicine, continuous backup, and the ability to scale beyond 100,000 patients.

5. Requirements Analysis

5.1 Functional Requirements

Area	Requirements
Patient Management	Register patient; update profile; manage medical history.
Appointment Management	Book, cancel, and reschedule appointments; manage doctor availability.
Electronic Health Records	Create records; update treatment history; view medical reports.
Doctor Services	Diagnosis management; prescription management; consultation management.
Telemedicine	Video consultation; online prescription; consultation session management.
Billing	Generate bills; process payments; maintain billing history.
Notifications	Appointment reminders; prescription alerts; health recommendations.
Analytics & AI	Health insights; disease-risk prediction; aggregate analytics.
Wearables	Receive and process health data from wearable/IoT devices.
Administration	Manage hospital resources, staff, roles, and operational information.

5.2 Non-Functional Requirements

Requirement	Target / Design Consideration
Scalability	Support 100,000+ patients and high concurrent usage.
Availability	24×7 access with redundant service instances and health checks.
Security	MFA, RBAC, TLS, encryption at rest, audit logging.
Performance	Fast EHR retrieval, caching, indexed queries, low-latency APIs.
Reliability	Replication, continuous backup, retry mechanisms, fault isolation.
Maintainability	Independent services, modular APIs, automated deployment.
Privacy	Least-privilege access and controlled sharing of medical information.

6. Architectural Design Approach

Microservice Architecture is selected because the proposed system is an enterprise healthcare platform with multiple independently evolving functions such as EHR, appointments, telemedicine, billing, notifications, analytics, and wearable integration. Each major business capability is implemented as an independently deployable service. An API Gateway provides a common entry point, while a message broker supports asynchronous processing.

6.1 Why Microservices Instead of a Single Layered Deployment?

Factor	Layered Architecture	Microservice Architecture
Deployment	Usually deployed as a larger unit.	Services can be deployed independently.
Scaling	Often scales the whole application.	Individual services can be scaled according to demand.
Fault Isolation	Failure may affect larger application areas.	Failure can be isolated to a service.
Technology Evolution	More tightly coupled.	Services can evolve independently.
Operational Complexity	Lower.	Higher because of distributed infrastructure.
Suitability	Small/medium systems.	Large hospitals and cloud healthcare platforms.

7. Architecture Diagram

Figure 1 represents the proposed microservice architecture. Client applications communicate through the API Gateway. The gateway routes requests to business services. Each major service owns or accesses its relevant data store. The message broker supports asynchronous notifications and analytics. External services provide payment, messaging, video consultation, and wearable integration.

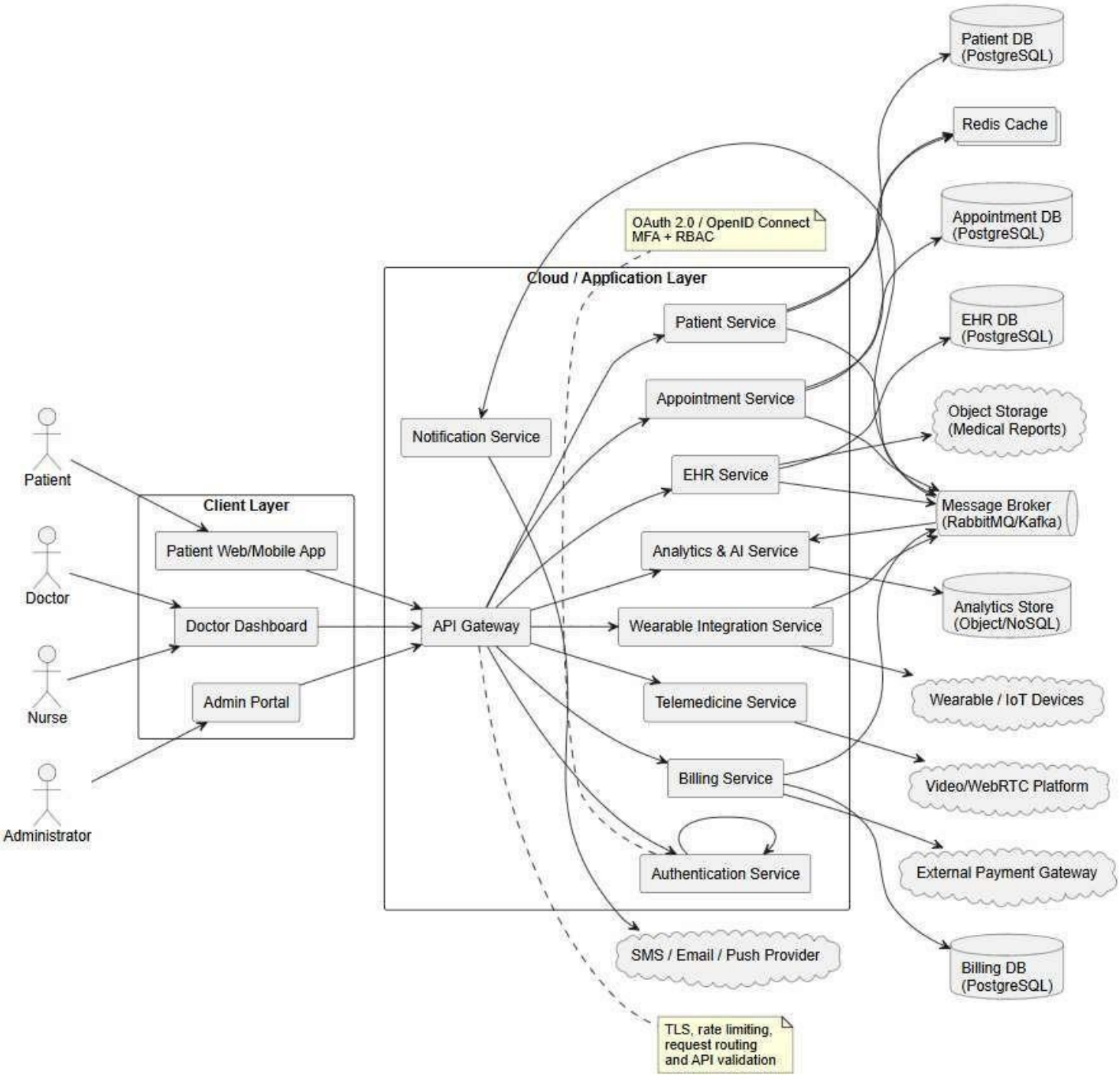


Figure 1: Smart Healthcare System – Microservice Architecture

8. Major Components and Service Description

Service / Component	Main Responsibility	Data / Integration
API Gateway	Single entry point; routing; request validation; rate limiting.	Routes to microservices.
Authentication Service	Login, token management, MFA, authorization support.	Identity store / OAuth 2.0 / OIDC.
Patient Service	Registration, profile, demographics, medical-history references.	Patient DB, Redis.
Appointment Service	Booking, cancellation, rescheduling, doctor availability.	Appointment DB, Redis.
EHR Service	Medical records, treatment history, reports, clinical information.	EHR DB, object storage.
Telemedicine Service	Video consultation sessions and consultation coordination.	WebRTC/video provider.
Billing Service	Bill generation, payment requests, transaction history.	Billing DB, payment gateway.
Notification Service	Appointment reminders, prescription alerts, recommendations.	Message broker, SMS/email/push provider.
Analytics & AI Service	Health analytics, dashboards, prediction models.	Analytics store.
Wearable Integration Service	Collect and normalize wearable/IoT health data.	IoT platform, message broker.
Message Broker	Asynchronous event delivery and decoupling.	RabbitMQ/Kafka.
Redis Cache	Caches frequently accessed data and session-related information.	In-memory cache.
Databases	Persistent storage separated by service/business capability.	PostgreSQL + analytics/object storage.

9. Key Interaction Flow

1. Patient logs in through the mobile/web application.
2. The request reaches the API Gateway over HTTPS.
3. The Authentication Service validates credentials and MFA and issues an access token.
4. The patient requests an appointment; the Appointment Service checks availability and creates the booking.
5. An appointment event is published to the message broker.
6. The Notification Service consumes the event and sends a reminder through SMS, email, or push notification.

7. During consultation, the Telemedicine Service creates a secure video session using a video/WebRTC platform.
8. The doctor records diagnosis, treatment, and prescription information through the EHR Service.
9. Billing Service generates the bill and communicates with the external payment gateway.
10. Relevant events can be consumed by the Analytics & AI Service for authorized aggregate analysis.

10. UML Deployment Diagram

The deployment diagram maps software artifacts and services to physical or virtual execution nodes. Client devices connect through a cloud load balancer to a Kubernetes cluster. Multiple service pods provide horizontal scalability. Databases are deployed in a managed database layer with replicas and backups. External integrations are separated from core application services.

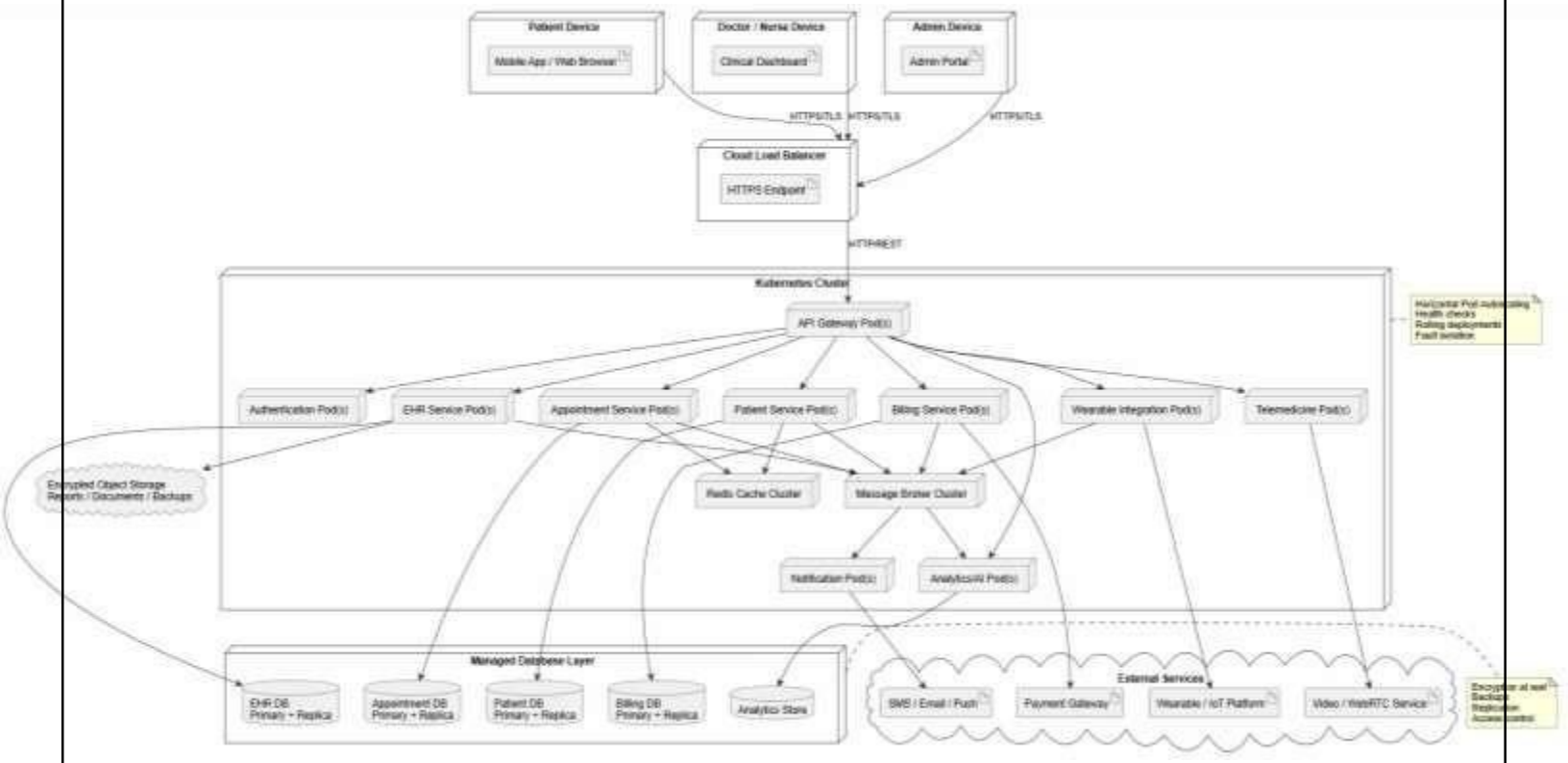


Figure 2: Smart Healthcare System – UML Deployment Diagram

11. Deployment Components

Deployment Node	Purpose
Patient Device	Runs mobile application or web browser.
Doctor/Nurse Device	Runs clinical dashboard.
Admin Device	Runs administrative portal.
Cloud Load Balancer	Distributes HTTPS traffic to application instances.
Kubernetes Cluster	Runs independently scalable service pods.

API Gateway Pods	Routes and validates API requests.
Service Pods	Run authentication, patient, appointment, EHR, telemedicine, billing, notification, analytics, and wearable services.
Message Broker Cluster	Supports asynchronous events and decoupled processing.
Redis Cluster	Provides low-latency caching.
Managed Database Layer	Persistent data storage with replication and backup.
Encrypted Object Storage	Stores medical reports, documents, and backups.
External Services	Payment, messaging, video, and wearable/IoT integrations.

12. Security Architecture

12.1 Authentication and Authorization

- OAuth 2.0 / OpenID Connect for secure identity and token-based access.
- Multi-Factor Authentication (MFA) for privileged and sensitive access.
- Role-Based Access Control (RBAC) for Patient, Doctor, Nurse, and Administrator roles.
- Least-privilege access to medical records and administrative functions.
- Short-lived access tokens and secure token validation at the API Gateway.

12.2 Data Protection

- TLS/HTTPS for data in transit.
- AES-256 or equivalent strong encryption for sensitive data at rest.
- Encrypted database backups and object storage.
- Secrets kept outside source code using a secure secrets-management mechanism.
- Restricted database/network access using private subnets and service-level permissions.

12.3 Audit Logging

- Patient record access.
- Prescription creation or modification.
- Login and authentication events.
- Changes to appointments and billing records.
- Administrative actions and permission changes.

13. Scalability and Performance Analysis

Technique	Application	Benefit
Horizontal Scaling	Multiple instances/pods of busy services.	Supports more concurrent users.
Load Balancing	Cloud load balancer before application cluster.	Distributes traffic and avoids single-server overload.
Database Replication	Primary/replica databases.	Improves availability and read

		scalability.
Caching	Redis for frequently accessed appointment/availability data.	Reduces database load and latency.
Asynchronous Messaging	Broker for notifications and analytics events.	Prevents slow secondary tasks from blocking user requests.
Database Indexing	Indexes patient/appointment lookup fields as appropriate.	Faster queries.
CDN	Static web assets and non-sensitive public content.	Improves content delivery.
Compression	API responses and web assets where appropriate.	Reduces bandwidth.
Autoscaling	Kubernetes Horizontal Pod Autoscaler.	Adds/removes instances according to demand.

14. Reliability and High Availability

- Run multiple instances of critical services so one instance failure does not stop the service.
- Use load balancing and health checks to route traffic only to healthy instances.
- Use database replication and automated backups.
- Use retries with controlled backoff for transient external-service failures.
- Use message queues so notification and analytics failures do not block core transactions.
- Use monitoring, centralized logs, and alerts for early fault detection.
- Use rolling deployments to reduce downtime during releases.
- Define recovery procedures and regularly test backup restoration.

15. Architecture Trade-offs

Advantage	**Trade-off / Risk**	**Mitigation**
Independent scaling	More infrastructure to manage.	Container orchestration and automation.
Fault isolation	Distributed failures can be harder to diagnose.	Centralized monitoring, tracing, and correlation IDs.
Independent deployment	Version compatibility must be managed.	API contracts and backward-compatible changes.
Service ownership of data	Cross-service queries become more complex.	APIs, events, and carefully designed read models.
Async processing	Events may be delayed or duplicated.	Idempotent consumers, retries, and dead-letter queues.
Cloud-native flexibility	Higher cloud/operational cost.	Autoscaling and right-sizing resources.

16. Design Assumptions

- The organization operates one logical healthcare platform that may serve multiple hospitals or departments.
- Users access the system through web or mobile clients.

- PostgreSQL is used for transactional data; an analytics store is used for large analytical workloads.
- Medical documents are stored in encrypted object storage rather than directly in application servers.
- External payment, messaging, video, and wearable providers expose secure APIs.
- The system uses HTTPS/TLS for external and internal API communication where supported.
- RBAC and MFA are implemented for appropriate roles and risk levels.
- Only authorized users can access patient information according to organizational policies.
- AI predictions are decision-support features and are not treated as a replacement for professional clinical judgment.
- Healthcare compliance requirements are treated as design constraints; the exact regulatory obligations depend on deployment geography and organization.

17. Methodology

11. Requirement Analysis – identify stakeholders, functions, and non-functional requirements.
12. Architectural Design – select microservices and define service boundaries.
13. UML Modeling – create architecture and deployment diagrams.
14. Security Design – define authentication, authorization, encryption, and auditing.
15. Scalability Analysis – identify bottlenecks and horizontal/vertical scaling strategies.
16. Documentation – record components, assumptions, trade-offs, and justification.

18. Architecture Justification

The microservice architecture is appropriate because the Smart Healthcare System contains business capabilities with different workloads and availability requirements. Appointment scheduling may experience periodic spikes, telemedicine requires low-latency communication, EHR requires strong consistency and controlled access, while analytics can execute asynchronously. Separating these capabilities allows each service to be scaled and maintained independently. The API Gateway simplifies client access and centralizes cross-cutting controls such as routing and rate limiting. A message broker reduces coupling for notifications and analytics. Database separation reduces direct coupling between services and supports independent data management. Kubernetes provides a practical cloud-native deployment model with health checks, rolling deployment, and horizontal scaling.

19. Security and Scalability Summary

Security is addressed through defense-in-depth: MFA and OAuth 2.0/OIDC at the identity layer, RBAC at the authorization layer, TLS for communication, encryption at rest, secure secrets management, audit logging, and restricted network/database access. Scalability is achieved through load balancing, horizontal service scaling, caching, database replication, asynchronous processing, indexing, and autoscaling. Reliability is strengthened through redundancy, backups, retries, health checks, and fault isolation.

20. Conclusion

The proposed Smart Healthcare System uses a microservice architecture and cloud-oriented deployment model to provide scalable, secure, reliable, and maintainable healthcare services. The architecture separates patient management, appointments, EHR, telemedicine, billing, notifications, analytics, authentication, and wearable

integration into focused services. The deployment model uses load balancing, containerized service instances, managed databases, caching, asynchronous messaging, and external service integration. Security controls such as MFA, RBAC, TLS, encryption, and audit logging protect sensitive healthcare information. Overall, the design demonstrates how UML and modern software architecture principles can be applied to an enterprise healthcare platform.